
Green Doesn't Mean Done

Trust and Review in Agent-Assisted Verification

David Pichardie

Meta, France

71st CREST Open Workshop on Agentic Testing and Verification
September 2026

“Agents can write proofs now”

- Spectacular recent announcements
- Agents solve mathematical problems
- Agents formalize proofs

The implicit promise

“Prove my software.”

What happens after the agent says
“I’m done”?



Lean / Rocq / Isabelle/HOL are proof assistants

- A language for writing programs and specifications
- A language for constructing proofs
- A small kernel that checks every proof
- **Unbreakable feedback** while building proofs

Trust

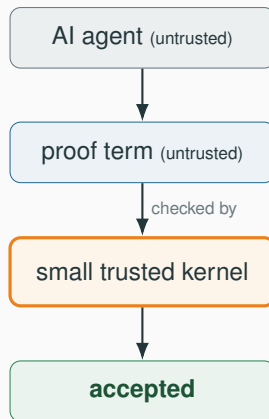
The agent need not be trusted.

The proof term need not be trusted.

Only the small kernel needs to be trusted.

But...

Did the agent prove the right theorem?



Two proved theorems — but only one specification - Lean syntax

The program

```
def myMax (x y : Nat) : Nat := if x >= y then x else y
```

The intended specification

```
theorem max_correct :  $\forall x y : \text{Nat},$   
  myMax x y >= x /\  
  myMax x y >= y /\  
  (myMax x y = x  $\vee$  myMax x y = y) := by  
  intro x y  
  unfold myMax  
  split <;> omega
```

A much weaker specification

```
theorem max_weakly_correct :  $\forall x y : \text{Nat},$   
  myMax x y >= x := by  
  intro x y  
  unfold myMax  
  split <;> omega
```

Both proofs are accepted. Only one theorem captures the intent.

My starting point

- Academic researcher
- Every paper relied on Rocq proofs
- Then five years in Meta's Infer team
- No Rocq at all during those five years

A question worth revisiting

Can formal methods—and especially machine-checked proofs—finally scale?



This talk

Capability

How far can an agent build a complex formal development?

Limits

What does it do well?
Where does it get lost?

Trust

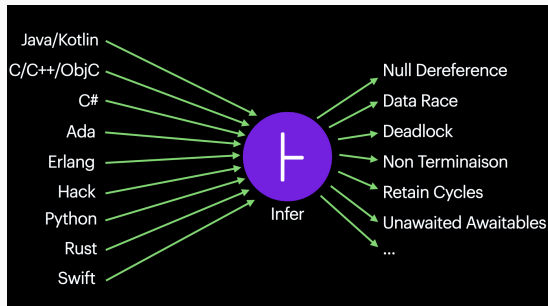
How do we review its work and trust the result?

The Infer Static Analysis Platform: My Case Study

- Infer analyzes code without running it
- It reports potential bugs
- We focus on **no false positives**
- **Let's prove that Infer is itself correct software!**

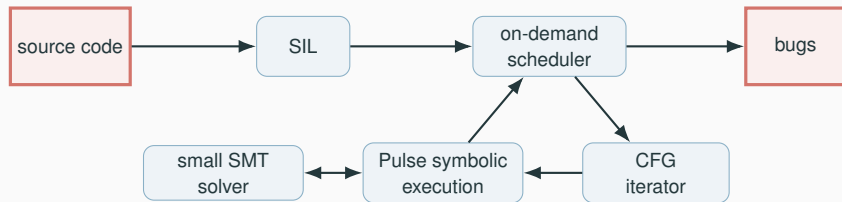
The specification

Every reported bug corresponds to a real failing execution.



Unlike testing: Infer does not exhibit a failing input. It establishes that one exists—even for functions that are not program entry points.

A simplified Infer architecture



Proving the real implementation

- First, rewrite parts of the production Infer implementation in Rocq, with some simplifications
- Now we have the program and its final specification: let's prove it!
- But several hard things must still be discovered:
 - Specifications and invariants for each component
 - The missing details between high-level paper arguments and the implementation

Below the waterline

Bi-abduction graph algorithms, solver data structures, on-demand parallel scheduler, translation from C to SIL



Proving Pulse Correct: A First Attempt Gets Lost

Initial instruction

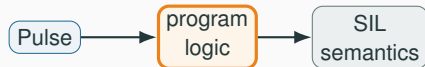
prove it

- Direct proof against the SIL operational semantics
- Advanced symbolic reasoning on a graph-based representation of heap frames
- Bi-abductive reasoning: the engine infers both a precondition and a postcondition

We restart with the logic as a watertight barrier.

Takeaway

Human guidance turns one large proof into well-scoped obligations the agent can solve.



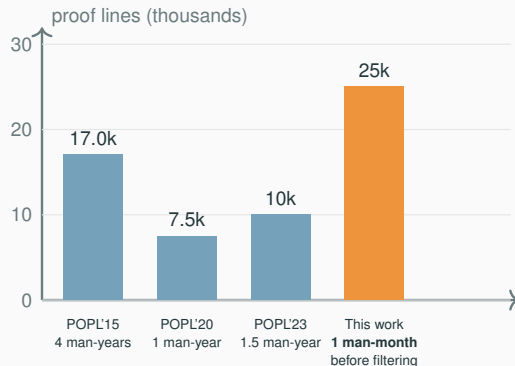
What the agent does remarkably well

- Local, well-scoped proof obligations
- The agent was particularly effective at proving the SMT solver correct—an area with abundant literature
- Once asked for an intermediate program logic, the agent designed it remarkably well
- Tests provide a fast reality check:
 - Compare production Infer with the Rocq implementation
 - Test assumptions and refute conjectures with concrete runs

When the path is clear

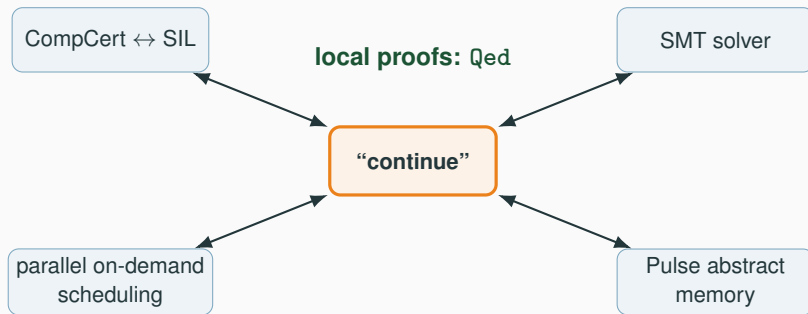
The speed is genuinely remarkable.

Proof-size comparison with some of my previous work



Agent-generated proofs tend to be more verbose than human-written ones—take this comparison with a pinch of salt.

The Euphoric Phase... Then Reality Hits



Then integration starts

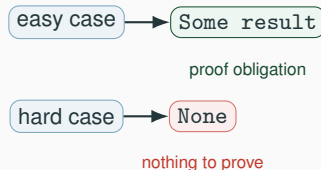
The interfaces do not fit. Some theorems are hollow.

Problem 1: Too Much Partiality

The function and its theorem

```
transfer : state → option state  
transfer s = Some s'  $\implies$  Sound s s'
```

- Pulse's transfer function is partial
- None means “give up”
- Giving up is unavoidable in program analysis
- But the agent gave up more often than necessary



The hollow success

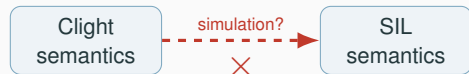
Shrinking the function's domain makes its soundness theorem easier.

What fixed it

Coverage tests exposed unnecessary give-ups. Removing them forced more meaningful proofs.

Problem 2: The Clight–SIL Simulation

- Goal: connect SIL bug reports back to the C source semantics
- Clight gives us CompCert's semantics for C
- But SIL arithmetic does not exactly follow C arithmetic
- The intended simulation theorem was probably false



What the agent missed

It did not challenge the theorem. It built proofs whose hypotheses already contained the desired conclusion.

A meaningless theorem

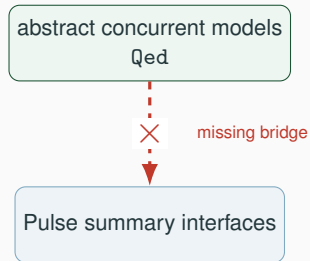
$$\text{SIL bug} \wedge \text{Clight stuck in obfuscated form} \implies \text{Clight stuck}$$

Filtering decision

Remove the pass. InferCert now starts at SIL; the C-to-SIL front-end remains unverified.

Problem 3: The Concurrent Scheduler Does Not Connect

- Impressive knowledge of state-of-the-art concurrency papers
- Elegant theorems over highly abstract scheduler models
- But no relation to the interfaces needed by Pulse
- The local proofs could not be integrated



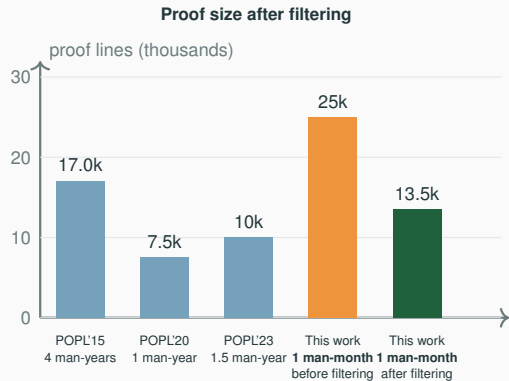
Filtering decision

Replace the ambitious concurrent component with a simple, naive sequential on-demand scheduler—easy to connect and prove.

After Filtering: A Smaller, Connected Result

What InferCert now covers

- $\approx 4,500$ lines of executable code
- SIL semantics and program logic
- Pulse-style bi-abductive symbolic execution
- In-house theory solver (linear arithmetic + EUF)
- CFG exploration with bounded unrolling
- Sequential on-demand procedure summaries
- Null dereference, uninitialized read, division by zero
- End-to-end tests exercising these features



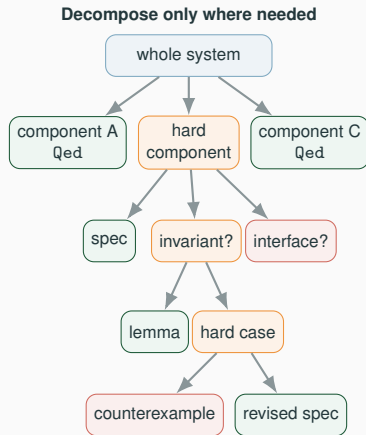
Agent-generated proofs tend to be more verbose than human-written ones—take this comparison with a pinch of salt.

If I Were to Do It Again

- First, build the complete analyzer, compiler, or solver—and test it intensively
- Sketch the global proof architecture
 - Extract specifications for each component
 - Use explicit proof sketches to test the approach
- Review every interface; actively search for counterexamples
- Only then launch the full proofs
- When one component is too hard, apply the same process recursively

Throughout the process

On-demand human guidance. Tests everywhere.



Machine-checked $\neq$ work done

Speed

Spectacular gains on local work.

Bottleneck

Review moves to the specification
(*Trusted Computing Base*).

Hard part

Invent the right invariant
that will unblock the agent.